



COMSATS University, Islamabad Pakistan

Campus Connect

Software Requirements Specification

By

Rukhsar Fatima CIIT/SP24-BSE-045/ISB

Supervisor

Atique Ahmad Zafar

Bachelor of Science in Software Engineering (2024-2028)

Table of Contents

1	Chapter 1: Introduction and Problem Definition	3
1.1	Overview of the Project	3
1.2	Vision Statement	3
1.3	Problem Statement	3
1.4	Problem Solution	3
1.5	Objectives of the Proposed System	4
1.6	Scope	4
1.6.1	Limitations/Constraints	5
1.7	Modules	5
1.7.1	Module 1: Module Name	Error! Bookmark not defined.
1.8	Related System Analysis/Literature Review	Error! Bookmark not defined.
1.8.1	Literature Review	7
1.8.2	Related System Analysis	7
1.9	Tools and Technologies	8
1.10	Project Contribution	Error! Bookmark not defined.
1.11	Relevance to Course Modules	Error! Bookmark not defined.
2	Chapter 2: Requirement Analysis	11
2.1	User Classes and Characteristics	Error! Bookmark not defined.
2.2	Operating Environment	Error! Bookmark not defined.
2.3	Design and Implementation Constraints	Error! Bookmark not defined.
2.4	Use case Diagram	Error! Bookmark not defined.
2.5	Requirement Identifying Technique	Error! Bookmark not defined.
2.6	Functional Requirements	Error! Bookmark not defined.
2.6.1	Recommended Format:	Error! Bookmark not defined.
2.7	Non-Functional Requirements	1
2.7.1	Reliability	2
2.7.2	Usability	2
2.7.3	Performance	2
2.7.4	Security	2
2.8	External Interface Requirements	3
2.8.1	User Interfaces Requirements	3
2.8.2	Software interfaces	3
2.8.3	Hardware interfaces	4
2.8.4	Communications interfaces	4
	Appendix A	Error! Bookmark not defined.
	Example: Context Diagram	Error! Bookmark not defined.
	Example: User Classes and Characteristics	Error! Bookmark not defined.
	Example: Use case Diagram	Error! Bookmark not defined.
	Syntax for Use case Diagram	Error! Bookmark not defined.
	Example: Mockup-Based Requirement Analysis	Error! Bookmark not defined.
	Mockup M1 – Login Page	Error! Bookmark not defined.
	Functional Requirements Derived from Mockup M1 – Login Page	Error! Bookmark not defined.
	Mockup M2 – Dashboard Page	Error! Bookmark not defined.
	Functional Requirements Derived from Mockup M2– Dashboard Page	Error! Bookmark not defined.

List of Figures

Figure 4.3 Assignee Dashboard	Error! Bookmark not defined.
Figure A-1: Context diagram of the Cafeteria Ordering System...	Error! Bookmark not defined.
Figure A-2: Use Case Diagram of an Appointment System.....	Error! Bookmark not defined.

1 Chapter 1: Introduction and Problem Definition

1.1 Overview of the Project

Campus Connect is a university-focused digital platform designed to help students manage common daily activities within a trusted and secure environment. The system integrates multiple student-oriented services into one centralized platform, including a student marketplace, ride-sharing coordination, student verification, and a review system. The purpose of the project is to improve communication, accessibility, trust, and convenience among university students.

University students often face challenges such as finding affordable academic resources, arranging transportation, and identifying trustworthy individuals for online interactions. Existing solutions such as WhatsApp groups, Facebook Marketplace, and public ride-sharing platforms are fragmented, unorganized, and frequently lack verification mechanisms. These issues increase the risk of scams, fake profiles, and unreliable transactions.

Campus Connect addresses these challenges by providing a closed-loop system restricted to verified university students only. Students can securely buy and sell items, coordinate rides with fellow students, and interact through verified profiles and review systems. The platform aims to create a safer and more organized digital ecosystem specifically designed for university communities.

The proposed system will be developed using modern web and mobile technologies including React JS, React Native, Node.js, Express.js, MongoDB, Firebase Authentication, and Google Maps API. The system is intended to operate as a responsive web and mobile application accessible to students anytime and anywhere with an internet connection.

1.2 Vision Statement

For university students who need a trusted and centralized platform for transportation coordination, academic item trading, and secure student interaction, **the** Campus Connect **is** a university-focused web and mobile application **that** enables verified students to share rides, buy and sell academic resources, search for accommodations, and communicate safely within their campus community. **Unlike** existing informal platforms such as WhatsApp groups, Facebook Marketplace, and OLX, **our product** provides a secure closed-loop environment with university verification, trusted user profiles, organized listings, and integrated communication features that improve safety, reliability, and convenience for students.

1.3 Problem Statement

University students frequently face difficulties in managing everyday campus-related activities such as transportation, accommodation searching, and buying or selling academic materials. Currently, most students rely on informal communication channels such as WhatsApp groups, Facebook pages, and public online marketplaces to coordinate these tasks. These platforms are not specifically designed for university communities and therefore lack verification, security, organization, and trust.

Students often struggle to find reliable ride-sharing partners travelling on the same routes, resulting in increased transportation costs, wasted time, and safety concerns. Similarly, students searching for academic books, calculators, laptops, and hostel items frequently encounter fake listings, delayed responses, and unverified sellers on existing platforms.

Another major issue is the absence of a trusted student verification mechanism. Existing public platforms allow anonymous or fake accounts, increasing the risk of scams, fraud, and unreliable interactions. Due to the lack of a centralized university-specific system, students spend unnecessary time searching for resources and coordinating with multiple unrelated groups.

1.4 Problem Solution

Campus Connect provides a centralized solution specifically designed for university students. The system combines essential student services into a single platform accessible only to verified users belonging to the university.

The proposed solution includes:

- A student marketplace for buying and selling academic and personal items
- A ride-sharing module for coordinating transportation among students
- A student verification system to ensure trusted interactions
- A review and rating system to improve accountability and transparency

The system uses university email verification and student ID verification to restrict access to genuine students only. By integrating all services into one platform, Campus Connect improves organization, convenience, safety, and communication within the university community.

The review and rating system further enhances reliability by allowing students to evaluate their experiences with other users. This helps reduce fraudulent activities and encourages responsible behavior on the platform.

1.5 Objectives of the Proposed System

BO-1: To provide a centralized university-specific platform for student interaction and resource sharing.

BO-2: To reduce transportation costs and travelling difficulties for students through an organized ride-sharing system.

BO-3: To provide a secure and trusted marketplace for buying and selling academic and personal items.

BO-4: To improve platform security and reliability through student verification using university credentials.

BO-5: To reduce the risk of scams, fake profiles, and fraudulent listings through administrator moderation and review systems.

BO-6: To reduce the time students spend searching for transportation, accommodation, and academic resources.

BO-7: To provide an easy-to-use web and mobile platform accessible to students anytime and anywhere.

1.6 Scope

The scope of the Campus Connect project is to develop a centralized digital platform that helps university students manage daily logistical needs in a secure and organized environment. The system will be accessible as both a web and smartphone application. Students can register using university email verification and submit student ID for campus affiliation confirmation. Once verified, they can post, browse, request, and confirm rides within the university, enabling efficient ride-sharing and reducing commuting costs. The

Campus Connect SRS

platform will provide verified hostel and accommodation listings, allowing students to view, and contact owners or residents easily. A dedicated marketplace will let students buy, sell, and exchange books, technology, and academic materials with verified peers. Real-time messaging and notifications will keep students updated on rides, accommodations, and marketplace activity. Users can rate and review rides and transactions to ensure trust and transparency. Administrators will manage user verification, moderate content, and monitor platform activity for security. Access will be restricted until verification is completed, ensuring only authorized users participate. The system will reduce time and effort spent coordinating rides, finding accommodations, and exchanging resources. Overall, Campus Connect aims to create a secure, efficient, and convenient ecosystem for students to coordinate transportation, accommodation, and academic resources

1.6.1 Limitations/Constraints

LI-1: Online payments for marketplace transactions are not supported in the current version; users must coordinate payments offline.

LI-2: Ride-sharing matches are basic and do not include route optimization or AI-based suggestions at this stage.

LI-3: Accommodation bookings cannot be completed directly through the platform; students must contact owners manually.

LI-4: The current version supports only a single university; multi-university support is planned for future releases.

LI-5: Personalized recommendations for rides, accommodations, or marketplace items are not yet implemented.

LI-6: The platform requires an active internet connection and does not support offline access.

LI-7: An informal social networking feature, similar to Discord, for students to interact and network within the same university is not included in the current release and may be added in future updates.

1.7 Modules

1.7.1 Client Application (Student App)

Module 1: User Registration and Authentication

FE-1: Register user accounts using university email verification

FE-2: Login using secure email and password authentication

FE-3: Enable OTP-based account verification

FE-4: Support password recovery and reset functionality

FE-5: Allow users to manage login sessions and logout

FE-6: Maintain secure authentication records for users

1.7.2 Module 2: Campus Verification

FE-1: Submit university email for campus verification

FE-2: Upload student ID card for identity verification

FE-3: View verification status within the application

FE-4: Receive approval or rejection notifications

FE-5: Resubmit verification documents if rejected

FE-6: Restrict platform access until verification is completed

1.7.3 Module 3: Ride Sharing System

FE-1: Post ride offers including route, time, and available seats

FE-2: Request rides from other students

FE-3: Browse available rides within the campus community

FE-4: Confirm or cancel ride bookings

FE-5: Rate and review ride-sharing experiences

FE-6: View ride history and booking details

1.7.4Module 4: Hostel Search and Accommodation Listings

FE-1: Browse available hostel listings near the campus

FE-2: View hostel details including rent, facilities, and location

FE-3: Filter hostel listings by price, distance, and amenities

FE-4: Contact hostel owners or student residents

FE-5: Save preferred hostel listings

FE-6: Report inaccurate or fraudulent hostel listings

1.7.5Module 5: Student Marketplace

FE-1: Create listings for books or technology items for sale

FE-2: Browse available marketplace listings

FE-3: Search items by category, subject, or price

FE-4: View item details including descriptions and images

FE-5: Contact sellers for negotiations through messaging

FE-6: Mark items as sold or purchased

1.7.6Module 6: Messaging and Notifications

FE-1: Send and receive direct messages between users

FE-2: Enable real-time chat for ride sharing and marketplace transactions

FE-3: Receive notifications for ride requests and responses

FE-4: Receive alerts for new marketplace or hostel listings

FE-5: Notify users about verification status updates

FE-6: Display sponsored marketplace advertisements

FE-7: Apply small transaction fee for marketplace purchases

Admin Web Application

1.7.1 Module 7: User and Verification Management

FE-1: View and manage registered user accounts

FE-2: Review uploaded student ID verification documents

FE-3: Approve or reject student verification requests

FE-4: Suspend or deactivate suspicious user accounts

FE-5: Monitor user activity and login history

FE-6: Restore or reactivate suspended accounts

1.7.2 Module 8: Content Moderation and Fraud Prevention

FE-1: Review and moderate marketplace listings

FE-2: Monitor ride-sharing posts and activities

FE-3: Remove fraudulent or inappropriate listings

FE-4: Review user reports and complaints

FE-5: Block or warn users involved in suspicious activities

FE-6: Maintain logs of moderation and enforcement actions

1.8.1 Literature Review

Recent studies highlight the growing importance of university-focused digital platforms for improving student convenience, safety, and communication. Research shows that students increasingly depend on online systems for transportation coordination, academic resource exchange, and peer interaction.

According to research on campus ride-sharing systems, students often experience transportation difficulties due to high commuting costs and lack of coordinated travel arrangements. Ride-sharing applications designed specifically for educational institutions can significantly reduce transportation expenses and improve travel efficiency.

Studies on online student marketplaces indicate that peer-to-peer academic resource exchange helps reduce educational expenses and promotes sustainability through item reuse. However, public platforms often lack security and verification mechanisms, increasing the possibility of fraud and unreliable transactions.

Research on trust and verification systems demonstrates that identity verification and review-based reputation systems improve user confidence and reduce malicious activities in online communities. University-specific verification systems are particularly effective in maintaining platform reliability because they restrict access to authenticated students only.

The findings from these studies support the development of Campus Connect as a secure and centralized platform tailored to university students.

1.8.2 Related System Analysis

Several existing systems provide partial solutions related to marketplace services, ride-sharing, or communication platforms. However, most of these systems are either public platforms or lack university-specific verification and integration.

1. Facebook Marketplace

Facebook Marketplace allows users to buy and sell products online. Although widely used by students, it does not provide university-specific verification, making transactions less secure.

2. WhatsApp Groups

Students commonly use WhatsApp groups for ride coordination and buying/selling items. However, these groups are highly unorganized, difficult to manage, and lack search and filtering capabilities.

3. OLX

OLX is a public online marketplace used for item trading. While it supports buying and selling activities, it is not restricted to students and therefore lacks trust and accountability mechanisms for university communities.

4. Careem / Uber

Ride-sharing applications such as Careem and Uber provide transportation services but do not specifically support student-based ride coordination or cost-sharing among university students.

Campus Connect differs from these systems by integrating multiple student-oriented services into a single verified university platform.

Table 1.1 Related System Analysis with Proposed Project Solution

Application Name	Weakness	Proposed Project Solution
U.n.I (LUMS Student App)	Although U.n.I connects students within a university network, its functionality is limited and does not fully integrate transportation, accommodation, and resource exchange into a single ecosystem.	Campus Connect extends the concept by creating a comprehensive university resource management platform integrating multiple student services in one trusted environment.
Facebook / WhatsApp Student Groups	Information is scattered and unstructured, making it difficult to find reliable rides, accommodation, or resources. Users are not always verified which can lead to trust and security issues.	Campus Connect provides a structured platform with organized listings and verified university users, improving reliability and ease of access.
OLX / Facebook Marketplace	These are public marketplaces where sellers are unknown and transactions require significant verification effort. They are not specifically designed for students or academic resources.	Campus Connect offers a university-specific peer marketplace where verified students can buy or sell books, devices, and study materials safely.
InDrive / Yango rides	These services provide transportation but rides are usually individual and costly for students . They do not support coordination or cost-sharing between students traveling on similar routes.	Campus Connect enables student ride-sharing coordination , allowing students from the same university to share rides and reduce commuting costs.

1.9 Tools and Technologies

Table 1.1 Tools and Technologies for Proposed Project

Tools And Technologies	Tools	Version	Rationale
	Visual Studio Code	1.85+	IDE
	Figma	Latest	UI/UX design and prototyping
	React JS	18	Front-end Development

	GitHub	Latest	Version control and collaboration
	Node.js	20	Backend server environment
	MongoDB	7	Database for users, rides, listings, transactions
	Firebase Authentication	Latest	Secure user authentication and verification
	Google Maps API	Latest	Location services for rides and hostel search

1.10 Project Contribution

The Campus Connect App introduces several technical and conceptual contributions compared to existing student-focused platforms:

Secure Campus Verification reduces fraud and boosts trust by integrating student ID verification and university email validation to guarantee that only authorized students and hostel residents can access the system.

Integrated Ride Sharing System: Provides a ride-sharing platform tailored to the campus where students can post, request, and reserve rides with seat availability, time, and route, increasing convenience and lowering transportation costs.

Hostel Search and Accommodation Listings: Enhances efficiency and safety when locating lodging by offering verified, campus-restricted hostel listings with comprehensive details, filtering, and communication options.

Student Marketplace with Monetization: Provides a platform for the purchase, sale, and promotion of technology and books, along with features for premium listings, advertisements, and transaction tracking, allowing the ecosystem to be sustainable.

Messaging and Notification System: Improving usability and communication within the campus network with real-time chat and alerts for rides, listings, and verification updates.

Admin Dashboard for System Oversight: Assures operational effectiveness and security by giving administrators the ability to manage verifications, keep an eye on transactions, moderate content, and regulate monetization features.

Data-Driven Features: Enhances user experience and helps students make well-informed decisions by allowing them to monitor ride history, listing activity, and notifications on a single platform.

Unique Integration of Multiple Services: Unlike using several external platforms, this unique integration of multiple services reduces fragmentation and improves convenience by combining ride sharing, hostel search, marketplace, and messaging into a single campus-restricted ecosystem.

Contribution Impact:

This system turns the student experience into a safe, effective, and connected ecosystem by combining secure verification, monetization, real-time communication, and campus-specific services into a single platform. This improves usability, trust, and engagement while offering chances for long-term financial gain.

1.11 Relevance to Course Modules

- **Software Engineering:** Applied SDLC phases—requirement analysis, system design, and modular architecture—to build the university ecosystem platform.

- **Programming Fundamentals:** Used conditional statements, loops, functions, and problem-solving logic to implement ride-sharing, hostel listings, and marketplace features.
- **Object-Oriented Programming (OOP):** Utilized encapsulation, inheritance, and modular class design to organize application elements and manage users, rides, listings, and transactions.
- **Database Systems:** Designed structured databases with schemas and queries to manage users, rides, hostel listings, and marketplace items efficiently.
- **Data Structures and Algorithms (DSA):** Implemented efficient search, filter, and data management algorithms for rides, accommodations, and marketplace operations.
- **Artificial Intelligence (AI):** Future integration of recommendation systems to suggest rides, hostels, and marketplace items based on user preferences.
- **Information Security:** Applied secure authentication, campus verification, and fraud prevention to maintain a trusted student environment.

Chapter 2: Requirement Analysis

1. Requirement Analysis

Campus Connect, a university resource management system is designed to centralise transportation coordination, accommodation discovery, and peer-to-peer academic resource exchange for university students. The analysis covers user classes, operating environment, design constraints, use case modelling, functional requirements derived through Mockup-Based Requirement Analysis, non-functional requirements, and external interface requirements.

2.1 User Classes and Characteristics

User classes anticipated to use Campus Connect along with their key descriptions;

User Class	Description
Student (Primary User)	University students who register, verify their campus identity, post or request rides, browse and list items in the marketplace, and search for hostel accommodation. There are an estimated 500–2,000 potential students per university, expected to interact with the system 3–5 times per week. Approximately 70% will access via smartphone app and 30% via web browser. Students range from first-year undergraduates to postgraduates and include both on-campus and off-campus residents.
Administrator	A university staff member or designated platform moderator responsible for reviewing and approving student verification documents, moderating marketplace and ride-sharing content, managing user accounts, and generating usage reports. Administrators interact primarily through the Admin Web Application. A typical deployment has 1–3 administrators per university instance.
Hostel Owner / Landlord	An external party (individual or organisation) who lists hostel or shared-apartment accommodation near the campus. Hostel owners do not require campus verification but must register a basic account to post and manage listings. They interact through a simplified listing interface and respond to student enquiries via the in-app messaging system.

2.2 Operating Environment

The following constraints define the environment in which Campus Connect must operate correctly.

OE-1: The system shall operate correctly on the following web browsers: Google Chrome (version 110 and above), Mozilla Firefox (version 110 and above), and Microsoft Edge (version 110 and above). Mobile web access shall be supported on Safari for iOS 15+ and Chrome for Android 10+.

OE-2: The backend server environment shall run on Node.js v20 hosted on a cloud platform (e.g., AWS or Azure), ensuring geographic availability to all university users in Pakistan.

OE-3: The mobile application shall be compatible with Android 10+ and iOS 15+. The application shall be packaged as a React Native app or Progressive Web App (PWA) to support both platforms from a single codebase.

OE-4: The system shall use MongoDB v7 as its primary database, deployed in a managed cloud cluster with automated backups. The database servers shall be geographically located within or close to Pakistan to comply with data residency requirement

OE-5: The system shall require an active internet connection (minimum 1 Mbps) for all operations. Offline access is not supported in the current version.

2.3 Design and Implementation Constraints

The following constraints restrict the design and implementation choices available to the developers.

CON-1: The frontend shall be built using React JS v18 for the web application and React Native for the mobile application, as mandated by the project technology stack.

CON-2: The backend API shall be developed using Node.js v20 with the Express.js framework.

CON-3: User authentication shall be implemented using Firebase Authentication to provide secure, standards-compliant email and OTP-based verification without building a custom authentication server.

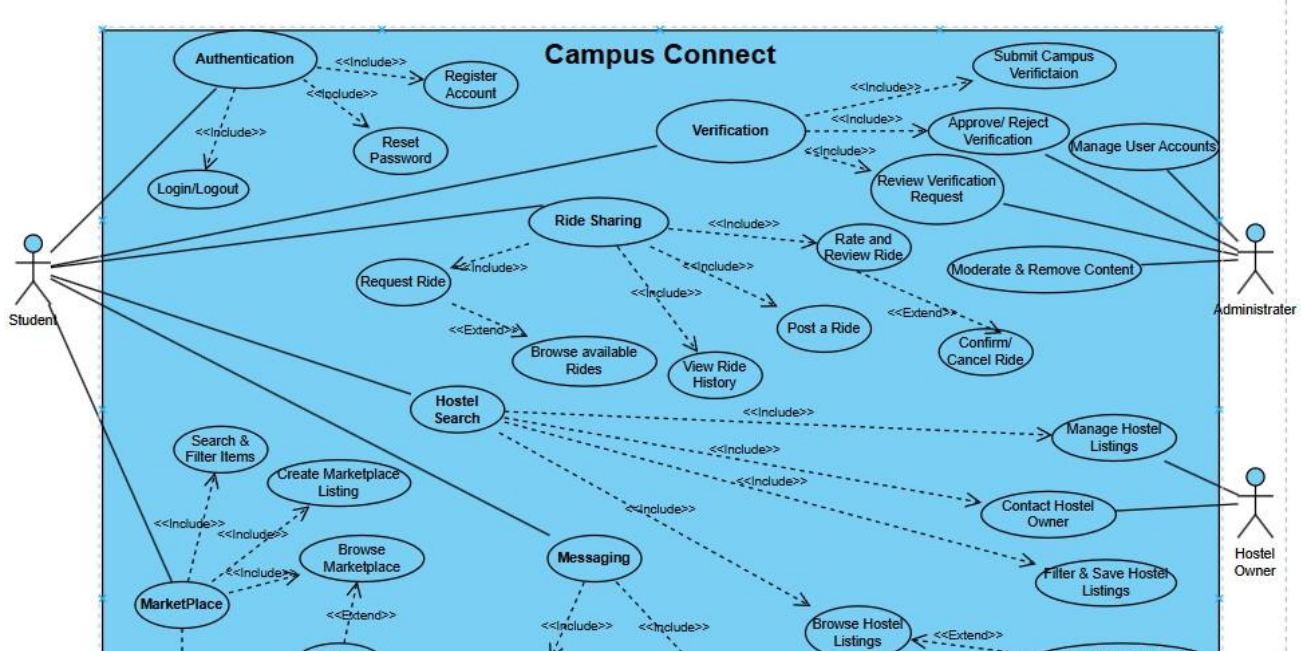
CON-4: The system shall use the Google Maps API (latest stable version) for all location-based features, including route display for ride sharing and proximity mapping for hostel listings.

CON-5: Online payment processing is not supported in the current version. All financial transactions between users (ride fares, marketplace payments) must be coordinated offline.

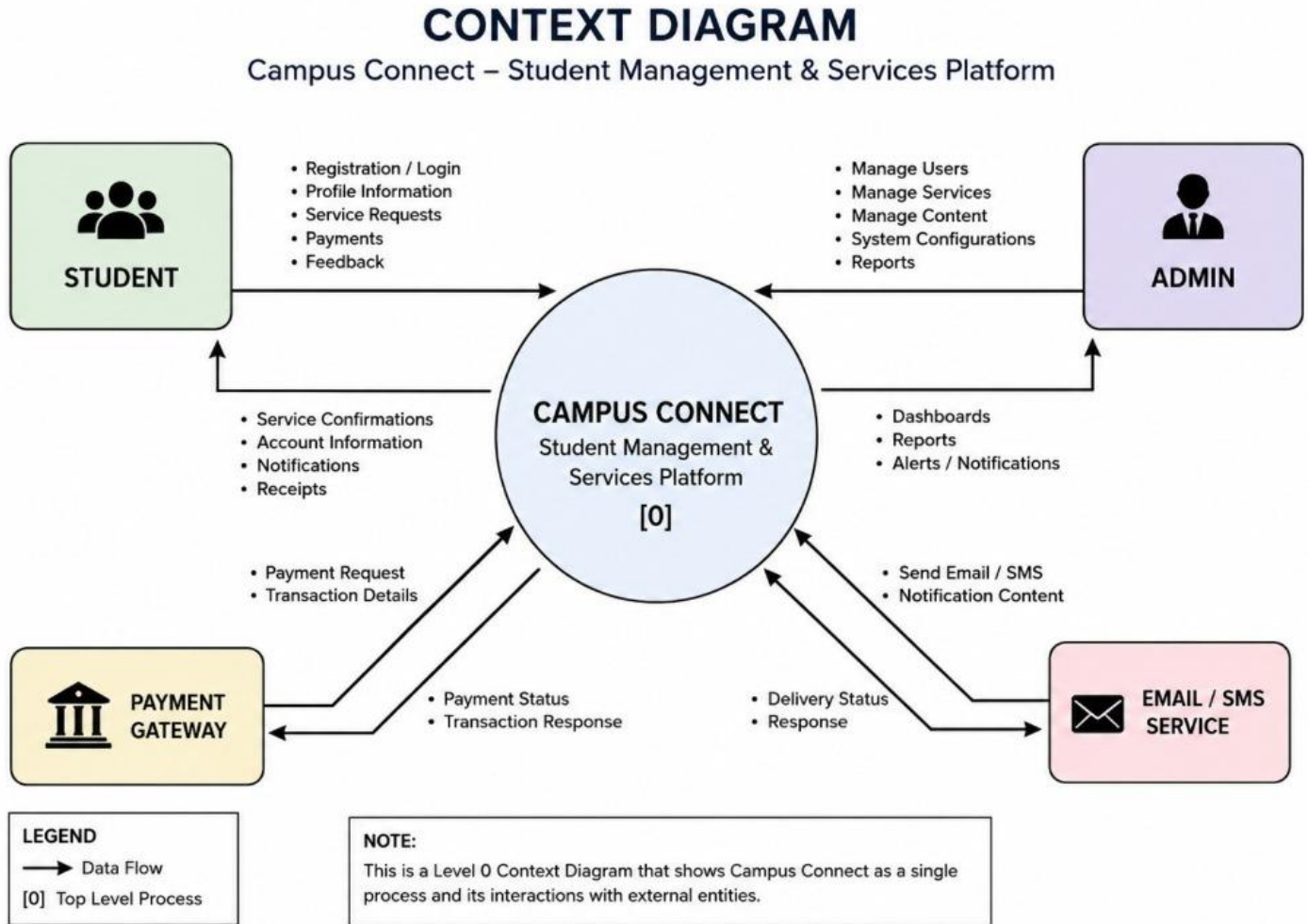
CON-6: The platform shall be restricted to a single university in the current release. Multi-university support requires a separate deployment and is planned for a future version.

2.4 Use case Diagram

Students from different universities have expressed the need for a centralized platform that is closed loop system and specific for a university that would allow the students from the same university to coordinate common dial needs within a structured and reliable environment. Providing this platform would improve coordination among students, increase the chances of finding the relevant opportunities quickly, and reduce unnecessary financial burden. Students could quickly identify ride partners travelling from the same route, locate available, safe and trustworthy accomodations near campus and connect with peers selling required books, tech devices and academic equipments. In



Context Diagram:



2.5 Requirement Identifying Technique

This section describes the technique used to identify and document the functional requirements of Campus Connect. These mockups visually represent how users will interact with each screen of the system. Functional requirements are then derived from each screen's interactive elements and expected behaviours.

Steps Followed:

- Core features were identified and broken down into individual screens.
- A Mockup ID was assigned to each screen (e.g., M1 for Registration, M2 for Campus Verification, etc.).
- Mockups of each screen were created in Figma, as shown in Section 1.6.
- Interactive elements on each screen were identified (buttons, forms, dropdowns, toggles, etc.).
- A functional requirement was derived from each element, describing what the system should do in response.

Table A-6: Functional Requirements Derived from Event–Response Table A-5

Event	Functional Requirement (FR-ID: Statement)	Business Rule
Marketplace Listing Created	FR5.3: The system shall publish the listing with status = ACTIVE immediately upon clicking 'Publish', making it visible and searchable to all verified students.	All mandatory fields must be valid before publishing.
Buyer Performs Search/Filter	FR6.2: The system shall provide a real-time search bar that filters listings by keywords matching item title, description, subject, or author name.	Search is case-insensitive.
Buyer Clicks 'Contact Seller'	FR6.5: The system shall allow a buyer to initiate a conversation with the seller by clicking 'Contact Seller' on the item detail view.	Buyers cannot contact their own listings.
Seller Marks Item as Sold	FR5.6: The system shall allow the seller to mark a listing as SOLD, removing it from active search results and archiving it in the seller's listing history.	Sold listings cannot be re-activated.
Seller Deletes Listing	FR5.5: The system shall allow the seller to permanently delete their own listing at any time.	Deleting a listing closes all related open chat threads.
Admin Removes Flagged Listing	FR-M6: The system shall allow the administrator to remove a reported listing, notify the seller with a removal reason, and log the moderation action.	None.

Backend Process Requirements

While mockups effectively capture frontend interactions, several system functionalities operate entirely at the backend — such as OTP generation, ride seat-count management, notification dispatch, and document verification workflows. To capture these processes, the Event–Response Table technique is used alongside the mockup-based approach. This ensures that all backend behaviours are fully documented in the requirements phase.

A.1 Event–Response Table: User Registration & Campus Verification

The following events are identified for the Registration and Campus Verification features:

- User submits the registration form.
- User enters the OTP for email verification.
- OTP validity timer expires.
- User submits campus verification documents.
- Administrator approves a verification request.
- Administrator rejects a verification request.

Table A-1: Event–Response Table for User Registration & Campus Verification

Event Name	Trigger	Source	System State (Before)	Response	System State (After)	Exception Handling
Registration Form Submitted	User clicks 'Register' button	External (Student)	No account exists	1. Validate email domain 2. Check for duplicate email 3. Hash password (bcrypt) 4. Create user record (status=UNVERIFIED) 5. Generate 6-digit OTP 6. Send OTP email via SMTP relay	Account created (UNVERIFIED); OTP sent	Reject if domain invalid or duplicate; show specific error message

OTP Entered Correctly	User submits matching OTP within 10 min	External (Student)	Account status = UNVERIFIED	1. Retrieve stored OTP hash 2. Verify match and check expiry 3. Set account status = EMAIL_VERIFIED 4. Invalidate OTP record 5. Redirect to Campus Verification screen	Account status = EMAIL_VERIFIED	None
OTP Entered Incorrectly (3× fails)	User submits 3 wrong OTPs	External (Student)	Account status = UNVERIFIED	1. Increment failure counter 2. Lock account for 15 minutes 3. Log lock event 4. Notify user of lock duration	Account locked for 15 min	Auto-unlock after 15 min; allow re-request of OTP
OTP Expiry Timer Fires	10 minutes elapse since OTP generation	System (Timer)	OTP active; account UNVERIFIED	1. Mark OTP as expired 2. Queue reminder email to user with 'Request New OTP' link	OTP invalid	Log expiry; user must request a new OTP
Verification Documents Submitted	User clicks 'Submit for Verification'	External (Student)	User has EMAIL_VERIFIED status	1. Validate file type & size 2. Store file in secure cloud storage 3. Create verification request (status=PENDING_REVIEW) 4. Notify admin dashboard	Verification request = PENDING_REVIEW	Reject invalid file type/size with error; preserve form data
Admin Approves Verification	Admin clicks 'Approve' on dashboard	External (Admin)	Verification = PENDING_REVIEW	1. Update user.verification_status = VERIFIED 2. Send push notification to student 3. Unlock all platform modules for user	User status = VERIFIED; all modules unlocked	None
Admin Rejects Verification	Admin clicks 'Reject' with reason	External (Admin)	Verification = PENDING_REVIEW	1. Update status = REJECTED 2. Record rejection reason 3. Send notification to student with reason 4. Enable 'Resubmit' option on student screen	Status = REJECTED; resubmit enabled	Log rejection count; flag account if rejected 3+ times

Table A-2: Functional Requirements Derived from Event–Response Table A-1

Event	Functional Requirement (FR-ID: Statement)	Business Rule
Registration Form Submitted	FR1.1: The system shall allow a new user to register by providing a full name, university email, and password.	Email must match an approved university domain.
OTP Entered Correctly	FR1.6: The system shall activate the user account only after the user correctly enters the 6-digit OTP sent to their email.	OTP is valid for 10 minutes only.
OTP Entered Incorrectly (3× fails)	FR1.6: The system shall lock the account for 15 minutes after 3 consecutive incorrect OTP entries.	Account auto-unlocks after 15 minutes.
OTP Expiry Timer Fires	FR1.5: The system shall invalidate an OTP that has not been used within 10 minutes of generation.	User must request a new OTP after expiry.

Verification Documents Submitted	FR2.3: The system shall submit the verification request and update the user status to PENDING_REVIEW upon clicking 'Submit for Verification'.	Re-submission is blocked while a request is PENDING_REVIEW.
Admin Approves Verification	FR2.6: The system shall unlock all restricted modules for the user once their verification status is set to VERIFIED.	None.
Admin Rejects Verification	FR2.7: The system shall notify the user with the rejection reason and enable resubmission upon administrator rejection.	Users may resubmit up to 3 times.

A.2 Event–Response Table: Ride Sharing System

The following events are identified for the Ride Sharing feature:

- A verified student posts a ride offer.
- A passenger requests a ride.
- The driver accepts or declines a ride request.
- A passenger cancels a confirmed booking.
- The driver cancels the entire ride.
- A passenger submits a post-ride rating.

Table A-3: Event–Response Table for Ride Sharing System

Event Name	Trigger	Source	System State (Before)	Response	System State (After)	Exception Handling
Ride Offer Posted	Driver clicks 'Post Ride' with all required fields completed	External (Student)	No active listing for this trip	1. Validate required fields 2. Validate departure date/time is in future 3. Create ride record (status=ACTIVE, seats_available=n) 4. Index listing for search	Ride listing = ACTIVE; visible to all verified students	Reject if date is past; highlight invalid fields
Passenger Requests a Ride	Passenger clicks 'Request Ride'	External (Student)	Ride status=ACTIVE; seats_available>0	1. Check ride status and seat count 2. Create booking record (status=PENDING) 3. Send push notification to driver	Booking = PENDING; driver notified	If seats=0: show 'No seats available'; refresh listing
Driver Accepts Request	Driver clicks 'Accept' on notification	External (Student/Driver)	Booking status=PENDING	1. Update booking status=CONFIRMED 2. Decrement ride.seats_available by 1 3. If seats=0: mark ride as FULL 4. Send confirmation	Booking=CONFIRMED; seat count decremented	None

				notification to passenger		
Driver Declines Request	Driver clicks 'Decline'	External (Student/Driver)	Booking status=PENDING	1. Update booking status=DECLINED 2. Notify passenger of decline	Booking=DECLINED; seat count unchanged	None
Passenger Cancels Booking	Passenger clicks 'Cancel Booking' and confirms	External (Student)	Booking status=CONFIRMED	1. Update booking status=CANCELLED 2. Increment ride.seats_available by 1 3. If ride was FULL: revert to ACTIVE 4. Notify driver	Booking=CANCELLED; seat restored	None
Driver Cancels Entire Ride	Driver clicks 'Cancel Ride' and confirms	External (Student/Driver)	Ride status=ACTIVE or FULL	1. Set ride status=CANCELLED 2. Set all related bookings=CANCELLED 3. Send notification to all affected passengers	Ride=CANCELLED; all passengers notified	Log cancellation; penalise repeat cancellations
Passenger Submits Ride Rating	Passenger submits 1–5 star rating after ride	External (Student)	Ride completed; booking=CONFIRMED	1. Validate ride is completed 2. Save rating record 3. Recalculate driver's average rating 4. Update driver profile display	Rating saved; driver average updated	Prevent duplicate ratings for same booking

Table A-4: Functional Requirements Derived from Event–Response Table A-3

Event	Functional Requirement (FR-ID: Statement)	Business Rule
Ride Offer Posted	FR3.6: The system shall validate all required fields and publish the ride listing upon clicking 'Post Ride'.	Departure date/time must be in the future.
Passenger Requests a Ride	FR4.4: The system shall allow a verified passenger to send a ride request to the driver by clicking 'Request Ride'.	A user cannot request their own ride listing.
Driver Accepts Request	FR-R3: The system shall update the booking status to CONFIRMED, decrement the available seat count by 1, and notify the passenger when the driver accepts a ride request.	None.
Driver Declines Request	FR-R4: The system shall update the booking status to DECLINED and notify the passenger when the driver declines a ride request.	None.
Passenger Cancels Booking	FR-R5: The system shall cancel the booking and restore the available seat count when a passenger cancels a confirmed booking.	None.
Driver Cancels Entire Ride	FR-R6: The system shall cancel the entire ride and all associated bookings, and notify all affected passengers, when the driver cancels a ride.	None.

Passenger Submits Ride Rating	FR-R7: The system shall allow a passenger to submit a 1–5 star rating for the driver after a completed ride, and update the driver's average rating on their profile.	Each passenger may rate a driver once per completed booking.
--------------------------------------	---	--

A.3 Event–Response Table: Student Marketplace

The following events are identified for the Student Marketplace feature:

- A verified student creates and publishes a marketplace listing.
- A buyer performs a search or applies filters.
- A buyer initiates contact with a seller.
- The seller marks an item as sold.
- The seller deletes a listing.
- An administrator removes a flagged listing.

Table A-5: Event–Response Table for Student Marketplace

Event Name	Trigger	Source	System State (Before)	Response	System State (After)	Exception Handling
Marketplace Listing Created	Seller clicks 'Publish' with all mandatory fields complete	External (Student)	No listing exists for this item	1. Validate required fields 2. Validate and resize/compress uploaded photos 3. Store photos in CDN 4. Create listing record (status=ACTIVE) 5. Index listing for full-text search	Listing=ACTIVE; searchable by all verified students	Reject if mandatory fields missing; preserve form data on error
Buyer Performs Search/Filter	Buyer types keyword or applies filter on browse screen	External (Student)	Listing index contains active listings	1. Tokenise search query 2. Query full-text search index with keyword + filters 3. Rank results by recency 4. Return paginated results (10 per page)	Filtered listing results displayed to buyer	Show 'No results' message if no matches found
Buyer Clicks 'Contact Seller'	Buyer clicks 'Contact Seller' on item detail view	External (Student)	Buyer viewing an active listing	1. Check existing conversation thread for this buyer-seller-listing combo 2. Create new thread if none exists 3. Attach listing reference to thread 4. Redirect buyer to messaging module with thread open	Chat thread open; listing context attached	Prevent contact on own listing; show error if listing was deleted
Seller Marks Item as Sold	Seller clicks 'Mark as Sold' and confirms	External (Student)	Listing status=ACTIVE	1. Update listing.status=SOLD 2. Remove listing from active search index 3. Archive listing in seller's history 4. Notify any saved-item watchers	Listing=SOLD; removed from search results	None

Seller Deletes Listing	Seller clicks 'Delete Listing' and confirms	External (Student)	Listing status=ACTIVE or DRAFT	1. Soft-delete listing (status=DELETED) 2. Remove from all search indexes 3. Close any open chat threads linked to this listing 4. Log deletion timestamp	Listing permanently hidden; related chats closed	Deletion is irreversible; warn user before confirmation
Admin Removes Flagged Listing	Admin clicks 'Remove Listing' after review	External (Admin)	Listing reported by user; status=ACTIVE	1. Set listing.status=REMOVED_BY_ADMIN 2. Remove from all search indexes 3. Notify seller with removal reason 4. Log moderation action with admin ID and timestamp	Listing=REMOVED; seller notified; action logged	Log repeated violations; flag seller account if 3+ removals

Table A-6: Functional Requirements Derived from Event–Response Table A-5

Event	Functional Requirement (FR-ID: Statement)	Business Rule
Marketplace Listing Created	FR5.3: The system shall publish the listing with status = ACTIVE immediately upon clicking 'Publish', making it visible and searchable to all verified students.	All mandatory fields must be valid before publishing.
Buyer Performs Search/Filter	FR6.2: The system shall provide a real-time search bar that filters listings by keywords matching item title, description, subject, or author name.	Search is case-insensitive.
Buyer Clicks 'Contact Seller'	FR6.5: The system shall allow a buyer to initiate a conversation with the seller by clicking 'Contact Seller' on the item detail view.	Buyers cannot contact their own listings.
Seller Marks Item as Sold	FR5.6: The system shall allow the seller to mark a listing as SOLD, removing it from active search results and archiving it in the seller's listing history.	Sold listings cannot be re-activated.
Seller Deletes Listing	FR5.5: The system shall allow the seller to permanently delete their own listing at any time.	Deleting a listing closes all related open chat threads.
Admin Removes Flagged Listing	FR-M6: The system shall allow the administrator to remove a reported listing, notify the seller with a removal reason, and log the moderation action.	None.

2.6 Functional Requirements

The Campus Connect system is built around 3 core features selected based on the primary student pain points identified during the requirements elicitation phase. These core features address the most frequent and impactful daily challenges faced by university students.

Selected Core Features

CF-2 Ride Sharing System	Students post, browse, request, confirm, and manage shared rides within the campus community.	fraud. Every other module depends on this. Students spend 1–3 hrs/day commuting. Ride sharing directly reduces cost and time — highest frequency use case per survey data.
CF-3 Student Marketplace	A peer-to-peer buy/sell platform for books, tech devices, and academic materials among verified students.	Reduces academic material costs by 20–30%. Students spend 3–4 days each semester locating resources — this directly solves that problem.

Justification

The three core features were chosen because:

- CF-1 (User Registration & Campus Verification) is the security backbone — without verified identities, the platform cannot enforce trust or safety.
- CF-2 (Ride Sharing) addresses the highest-frequency daily pain point (transportation) and offers immediate, measurable impact on student time and finances.
- CF-3 (Student Marketplace) solves the recurring academic resource problem at the start of each semester, reducing both cost and search time significantly.
- Together, these three features represent the unique value proposition of Campus Connect that differentiates it from open public platforms like OLX and WhatsApp groups.

2.6.1 MockUps and Derived Functional requirements

User Registration and Campus Verification



Register to CampusConnect



Upload Student Card

Registration number

Full name

Password

Confirm password

Create Account

**WELCOME
TO
CampusConnect!**



Verification Pending!

Check your University Email to verify your student account



Account Verified!!!

Please reload the page & Sign in again...



Verification Rejected...

Enter your credentials properly...

FR-ID	Functional Requirement	Source UC	Mockup Reference
FR-1.1	The system shall allow a new user to register by providing a full name, university email, and password.	UC-01	Screen M-1.1
FR-1.2	The system shall validate that the user's email belongs to a recognised university domain (e.g., @cuiisb.edu.pk) and reject non-university emails.	UC-01	Screen M-1.1
FR-1.3	The system shall send a 6-digit OTP to the registered email address within 30 seconds of account creation.	UC-01	Screen M-1.1
FR-1.4	The system shall activate the account only after the user successfully enters the correct OTP; after 3 failed attempts the account shall be locked for 15 minutes.	UC-01	Screen M-1.1
FR-1.5	The system shall allow a registered user to submit a campus verification request by providing their student roll number, program, batch, and a student ID photo.	UC-04	Screen M-1.2
FR-1.6	The system shall accept student ID uploads in JPG, PNG, or PDF format with a maximum file size of 5 MB; all other formats or larger files shall be rejected with an error message.	UC-04	Screen M-1.2
FR-1.7	The system shall set the user's verification status to PENDING_REVIEW upon successful document submission and notify the administrator of the new request.	UC-04, UC-05	Screen M-1.2
FR-1.8	The system shall restrict access to the Ride Sharing, Marketplace, Hostel, and Messaging modules until the user's verification status is VERIFIED.	UC-04	Screen M-1.2
FR-1.9	The system shall allow the administrator to approve or reject a verification request; upon rejection, the system shall notify the user with a reason and allow resubmission.	UC-05, UC-06	Admin Panel M-2.2

Ride Sharing System



Find a Rides

Ahmed Choudary
From CUI, Islamabad to Blue Area
3 seats avalibale

23 May, 2026 at 4pm
Rs 99 per seat

Decline

Accept

Search by Time or Location

POST A RIDE

Enter Your Location

Enter Starting point

Enter Destination point

Enter Date & Time

Available number of Seats

Fare per Seat

Additional Notes

Cancel

Post

FR-ID	Functional Requirement	Source UC	Mockup Reference
FR-3.1	The system shall allow a verified student to create a marketplace listing by providing a title, category, condition, description (max 500 characters), asking price, and up to 4 item photos.	UC-16	Screen M-3.1
FR-3.2	The system shall validate that all required fields (title, category, condition, price) are filled before publishing a listing.	UC-16	Screen M-3.1
FR-3.3	The system shall publish the listing with ACTIVE status immediately upon submission and make it visible to all verified students.	UC-16	Screen M-3.1
FR-3.4	The system shall allow the seller to edit, deactivate, or delete their own listing at any time; deleted listings are permanently removed from search results.	UC-16, UC-20	Screen M-3.1
FR-3.5	The system shall display active marketplace listings in a grid layout showing item photo, title, price, condition badge, and seller's verified status.	UC-17	Screen M-3.2
FR-3.6	The system shall provide a real-time text search bar that filters listings by item title, description keywords, subject, or author name.	UC-18	Screen M-3.2
FR-3.7	The system shall allow buyers to filter marketplace listings by category, price range (min–max), and item condition.	UC-18	Screen M-3.2
FR-3.8	The system shall allow a buyer to contact the seller through the in-app messaging system by clicking 'Contact Seller' on any active listing.	UC-19	Screen M-3.2
FR-3.9	The system shall allow the seller to mark an item as SOLD; once marked, the listing shall no longer appear in active search results.	UC-20	Screen M-3.1

Student Marketplace



SELL AN ITEM



Item Title

Category (Books/Tech/Stationary/other)

Condition

Description

Price



Upload High Quality Pictures

Save as Draft

Cancel

Publish



CompusConnect Marketplace



Rashyl

From CUI Islamabad wants to sell her

Dell Precision 8th generation Laptop

Category : Tech

Condition : 8/10

Price : 1,450,000

View Details



Search by Category



FR-ID	Functional Requirement	Source UC	Mockup Reference
FR-3.1	The system shall allow a verified student to create a marketplace listing by providing a title, category, condition, description (max 500 characters), asking price, and up to 4 item photos.	UC-16	Screen M-3.1
FR-3.2	The system shall validate that all required fields (title, category, condition, price) are filled before publishing a listing.	UC-16	Screen M-3.1
FR-3.3	The system shall publish the listing with ACTIVE status immediately upon submission and make it visible to all verified students.	UC-16	Screen M-3.1
FR-3.4	The system shall allow the seller to edit, deactivate, or delete their own listing at any time; deleted listings are permanently removed from search results.	UC-16, UC-20	Screen M-3.1
FR-3.5	The system shall display active marketplace listings in a grid layout showing item photo, title, price, condition badge, and seller's verified status.	UC-17	Screen M-3.2
FR-3.6	The system shall provide a real-time text search bar that filters listings by item title, description keywords, subject, or author name.	UC-18	Screen M-3.2
FR-3.7	The system shall allow buyers to filter marketplace listings by category, price range (min-max), and item condition.	UC-18	Screen M-3.2
FR-3.8	The system shall allow a buyer to contact the seller through the in-app messaging system by clicking 'Contact Seller' on any active listing.	UC-19	Screen M-3.2
FR-3.9	The system shall allow the seller to mark an item as SOLD; once marked, the listing shall no longer appear in active search results.	UC-20	Screen M-3.1

2.7 Non-Functional Requirements

This section defines the quality attributes of the Campus Connect system. All requirements are written in measurable, testable, and verifiable terms to ensure system reliability, usability,

performance efficiency, and security compliance. These requirements are independent of system functionality and focus on how the system behaves under operational conditions.

2.7.1 Reliability

NFR-R1: The system shall maintain a minimum uptime of 99.5% per month, excluding scheduled maintenance windows.

NFR-R2: In the event of system failure, critical services (login, ride posting, marketplace browsing) shall be restored within a maximum of 5 minutes recovery time ($RTO \leq 5 \text{ min}$).

NFR-R3: The system shall ensure data consistency such that no transaction (ride request, marketplace post, or hostel listing) is lost in case of abrupt system failure.

NFR-R4: The system shall implement automated error logging and monitoring mechanisms to detect system failures within 30 seconds of occurrence.

2.7.2 Usability

NFR-U1: A new user shall be able to complete core operations (login, browse marketplace, search rides/hostels) within 3 minutes of first use without training.

NFR-U2: All primary functions shall be accessible within a maximum of 3 navigation steps from the dashboard.

NFR-U3: The system shall maintain a consistent user interface across all modules using standardized components (buttons, forms, icons, and navigation layout).

NFR-U4: Error messages shall be user-friendly, clearly describing the issue and providing corrective guidance without technical jargon.

NFR-U5: The system shall support accessibility features such as readable font sizes, high contrast mode, and responsive layout for mobile and desktop devices.

2.7.3 Performance

NFR-P1: The system shall load all primary pages (dashboard, marketplace, rides, hostels) within 3 seconds under normal network conditions ($\geq 10 \text{ Mbps}$ connection).

NFR-P2: Search queries (marketplace items, rides, hostel listings) shall return results within 2 seconds for up to 10,000 records in the database.

NFR-P3: The system shall support at least 1,000 concurrent active users without significant performance degradation.

NFR-P4: Data updates (posting items, ride creation, hostel listing submission) shall be processed and reflected in the system within 2 seconds of submission.

2.7.4 Security

NFR-S1: Only verified university students shall be granted access to core system features through controlled authentication and authorization mechanisms.

NFR-S2: All communication between client and server shall be encrypted using HTTPS (TLS 1.2 or higher).

NFR-S3: The system shall enforce account protection mechanisms, including automatic detection of suspicious login attempts and temporary access restriction after repeated failures.

NFR-S4: User-generated content (posts, rides, reviews) shall be validated at both client and server levels to prevent injection attacks and malicious input.

NFR-S5: Sensitive user data shall be stored in encrypted form within the database to prevent unauthorized access.

2.8 External Interface Requirements

This section defines how the Campus Connect system interacts with users, external software services, hardware devices, and communication channels. These interfaces ensure interoperability, consistency, and secure data exchange across system components.

2.8.1 User Interfaces Requirements

UI-1: The system shall implement a responsive, mobile-first user interface compatible with desktop, tablet, and smartphone screens.

UI-2: The interface shall follow a consistent design system with standardized fonts, icon sets, color palette, and button styles across all modules.

UI-3: A persistent navigation bar shall be available in all authenticated screens, providing access to Marketplace, Ride Sharing, Hostel Listings, and User Profile.

UI-4: All input forms shall include real-time validation with inline error messages for incorrect or incomplete data.

UI-5: The system shall provide visual indicators (badges, icons, alerts) for system states such as verification status, ride confirmation, and listing approval.

UI-6: The system shall comply with basic accessibility standards, including readable font scaling, contrast optimization, and keyboard navigation support.

2.8.2 Software interfaces

SI-1: The system shall use a RESTful API architecture to enable communication between frontend (web/mobile) and backend services.

SI-2: The system shall use a MongoDB database for persistent storage of user profiles, marketplace items, ride data, and hostel listings.

SI-3: Authentication and session management shall be implemented using JWT (JSON Web Tokens) for secure user sessions.

SI-4: The system may integrate third-party services such as email/SMS APIs for verification, notifications, and alerts.

SI-5: The system backend shall be developed using Node.js with Express framework to handle business logic and API routing.

2.8.3 Hardware interfaces

HI-1: The system shall operate on standard computing devices including laptops, desktops, tablets, and smartphones without requiring specialized hardware.

HI-2: The system shall require only an internet-enabled device with a modern web browser for full functionality.

HI-3: No additional hardware peripherals are required for system operation.

2.8.4 Communications interfaces

CI-1: All system communication between client and server shall be performed over secure HTTPS protocol to ensure data confidentiality and integrity.

CI-2: The system shall support real-time or near real-time updates using WebSocket or equivalent asynchronous communication methods where applicable (e.g., ride updates, chat/notifications).

CI-3: The system shall support email-based notifications for account verification, activity alerts, and system updates.

CI-4: All API requests and responses shall follow JSON format for structured and standardized data exchange.